

Esame Programmazione di Sistema / API Programming

Data: 2023-01-16

QUESTIONS

Domanda 1

Nel caso di programmazione concorrente in Rust e memoria condivisa quali sono i principali costrutti e come vengono utilizzati per evitare la condizione di deadlock?

Domanda 2

Come si utilizza il sistema di gestione dei processi in Rust attraverso il pacchetto `std::process`? Quali sono le funzionalità offerte dalla libreria e come si gestiscono i processi figli nell'ambito del proprio programma?

Domanda 3

Definire il problema della dipendenze cicliche, includendo il motivo per cui si possa generare, ed indicare come in Rust sia possibile risolverle, facendo un esempio concreto.

Esercizio di Programmazione: `RankingBarrier`

Una barriera è un costrutto di sincronizzazione usato per regolare l'avanzamento relativo della computazione di più thread.

All'atto della costruzione di questo oggetto, viene indicato il numero N di thread coinvolti. Non è lecito creare una barriera che coinvolga meno di 2 thread.

La barriera offre un solo metodo, `wait()`, il cui scopo è bloccare temporaneamente l'esecuzione del thread che lo ha invocato, non ritornando fino a che non sono giunte altre $N - 1$ invocazioni dello stesso metodo da parte di altri thread: quando ciò succede, la barriera si sblocca e tutti tornano. Successive invocazioni del metodo `wait()` hanno lo stesso comportamento: la barriera è ciclica. Attenzione a non mescolare le fasi di ingresso e di uscita!

Una **`RankingBarrier`** è una versione particolare della barriera in cui il metodo `wait()` restituisce un intero che rappresenta l'ordine di arrivo: il primo thread ad avere invocato `wait()` otterrà 1 come valore di ritorno, il secondo thread 2, e così via. All'inizio di un nuovo ciclo, il conteggio ripartirà da 1.

Si implementi la struttura dati `RankingBarrier` a scelta nei linguaggi Rust o C++ '11 o successivi.

SOLUTIONS

Soluzione 1: Costrutti per Memoria Condivisa e Deadlock in Rust

I principali costrutti offerti da Rust per la programmazione concorrente con memoria condivisa sono:

- **Arc<T>** (Atomic Reference Counted): permette di avere multiple ownership di un dato allocato sull'heap tra più thread in modo thread-safe.
- **Mutex<T>**: garantisce la mutua esclusione, permettendo a un solo thread alla volta di accedere (sia in lettura che in scrittura) al dato interno T.
- **RwLock<T>**: permette accessi multipli in lettura simultanei, ma accesso esclusivo in scrittura.

Per quanto riguarda il **deadlock**, il compilatore Rust *non previene automaticamente* i deadlock a tempo di compilazione. Tuttavia, l'uso del sistema di ownership e di type system di Rust scoraggia alcuni pattern pericolosi e rende più visibile e locale il locking. Per evitare deadlock è necessario seguire buone pratiche:

- **Ordinamento dei lock**: se più lock devono essere acquisiti, si deve definire un ordine gerarchico stretto (es. prima A, poi B) per tutti i thread.
- **Lock a grana grossa**: minimizzare il numero di lock condivisi quando possibile, unendo dati correlati dietro a un solo Mutex.
- **Tempo di ritenzione minimo**: tenere i lock acquisiti per il minor tempo possibile ed in scope ridotti, in modo che vengano rilasciati (grazie al pattern RAII e alla chiamata a `drop()`) non appena il dato non è più necessario, evitando di invocare altre operazioni bloccanti tenendo un lock acquisito.

Soluzione 2: Gestione dei processi con `std::process`

Il pacchetto `std::process` in Rust fornisce gli strumenti necessari per spawnare nuovi processi figli e interagirvi. I concetti chiave sono:

- **Command**: è il builder pattern principale usato per configurare e lanciare i processi. Permette di specificare l'eseguibile (`Command::new("comando")`), passare argomenti (`.arg()`), modificare l'ambiente (`.env()`), la directory di lavoro (`.current_dir()`) e configurare stdin, stdout e stderr (`.stdin(Stdio::piped())`, ecc.).
- **Child**: una volta che un `Command` è stato eseguito con `.spawn()`, restituisce un handle al processo figlio di tipo `Child`.
- **Gestione del figlio**: sull'oggetto `Child` possiamo invocare metodi come `.wait()` (che blocca il thread corrente finché il figlio non termina e restituisce lo status di uscita) oppure `.kill()` (per forzarne l'arresto).
- **Interazione ed I/O**: tramite `spawn()`, i canali `stdin`, `stdout` e `stderr` del `Child` possono essere configurati come `Stdio::piped()`, permettendo al genitore di scrivere o leggere dai rispettivi flussi del processo figlio (es. `child.stdin.as_mut().unwrap().write_all(...)`).
- **Output diretto**: usando `Command::output()` il processo viene avviato, il genitore attende la fine, e cattura tutto l'output (stdout/stderr) contemporaneamente, restituendo un `Output`.

Soluzione 3: Dipendenze cicliche e soluzioni in Rust

Il problema delle **dipendenze cicliche** si presenta quando due o più tipi dipendono ricorsivamente tra loro per la gestione della memoria, formando un "ciclo di riferimenti". Nei linguaggi con *Garbage Collection* questo spesso viene gestito, ma in Rust (che usa il reference counting tramite `Rc` o `Arc`), un ciclo causa una **memory leak** (perdita di memoria): il conteggio dei

riferimenti dei nodi coinvolti non scenderà mai a zero e le loro aree di memoria (e le implementazioni di `Drop`) non saranno mai rilasciate. Questo si genera, ad esempio, in una struttura dati come un albero con riferimenti al genitore o una lista doppiamente concatenata, dove il nodo A punta al nodo B (tramite `Rc`) e il nodo B punta al nodo A.

Soluzione in Rust: Rust risolve le dipendenze cicliche fornendo i *Weak pointer* (`std::rc::Weak` o `std::sync::Weak`). Un Weak pointer si ottiene chiamando `Rc::downgrade(&rc_ptr)`. A differenza dei riferimenti "forti" (`Rc` o `Arc`), un riferimento debole non incrementa lo `strong_count`. Se lo `strong_count` scende a 0, la memoria del dato verrà deallocata, anche se il `weak_count` è > 0 . Per utilizzare un puntatore debole, bisogna tentare di "promuoverlo" temporaneamente a forte chiamando `.upgrade()`, che restituisce `Option<Rc<T>` (restituendo `None` se il valore è già stato deallocato).

Esempio concreto: In un albero, il genitore può possedere i figli con `Rc<RefCell<Node>`», mentre i figli possono avere un riferimento al genitore di tipo `Weak<RefCell<Node>`». In questo modo il figlio non tiene in vita forzatamente il padre, rompendo il ciclo.

```
1 use std::rc::{Rc, Weak};
2 use std::cell::RefCell;
3
4 struct Node {
5     value: i32,
6     parent: Weak<RefCell<Node>>,
7     children: Vec<Rc<RefCell<Node>>>,
8 }
9
10 fn main() {
11     let parent = Rc::new(RefCell::new(Node {
12         value: 1,
13         parent: Weak::new(),
14         children: vec![],
15     }));
16
17     let child = Rc::new(RefCell::new(Node {
18         value: 2,
19         parent: Rc::downgrade(&parent),
20         children: vec![],
21     }));
22
23     parent.borrow_mut().children.push(child);
24     // Non ci sono leak perche' 'child' ha solo un Weak pointer verso 'parent'
25 }
```

Soluzione Esercizio: RankingBarrier

Per implementare una barriera ciclica corretta, dobbiamo mantenere uno stato condiviso protetto da un `Mutex` e utilizzare una `Condvar` per bloccare/svegliare i thread.

Dato che la barriera è ciclica, è essenziale non mescolare thread di un "ciclo" o "generazione" precedente con quelli di un ciclo successivo. Per farlo, usiamo un contatore `generation` che viene incrementato quando un ciclo si completa. Ogni thread in arrivo salva la generazione corrente e attende sulla Condition Variable finché la generazione della barriera non cambia, indicando che la barriera è stata superata. Per restituire un ordine di arrivo, teniamo un contatore `count` (i thread arrivati finora), che funge da "rango" per l'estrazione. Esiste in alternativa l'utilizzo della struttura `std::sync::Barrier` standard, ma noi qua offriamo un'implementazione con `Mutex` e `Condvar` per finalità didattiche.

```
1 use std::sync::{Mutex, Condvar};
2
3 struct BarrierState {
4     count: usize,
```

```

5     generation: usize,
6 }
7
8 pub struct RankingBarrier {
9     n: usize,
10    state: Mutex<BarrierState>,
11    cvar: Condvar,
12 }
13
14 impl RankingBarrier {
15     pub fn new(n: usize) -> Self {
16         if n < 2 {
17             panic!("La barriera deve coinvolgere almeno 2 thread");
18         }
19         Self {
20             n,
21             state: Mutex::new(BarrierState {
22                 count: 0,
23                 generation: 0,
24             }),
25             cvar: Condvar::new(),
26         }
27     }
28
29     pub fn wait(&self) -> usize {
30         let mut state = self.state.lock().unwrap();
31
32         // Il thread registra il proprio arrivo
33         state.count += 1;
34         let rank = state.count;
35         let current_generation = state.generation;
36
37         if state.count == self.n {
38             // Sono l'ultimo thread: sblocco la barriera
39             // Ripristino count per il ciclo successivo
40             state.count = 0;
41             // Avanzo la generazione
42             state.generation = state.generation.wrapping_add(1);
43             // Sveglia tutti gli altri thread in attesa
44             self.cvar.notify_all();
45         } else {
46             // Non sono l'ultimo: attendo finche' la generazione non cambia
47             while state.generation == current_generation {
48                 state = self.cvar.wait(state).unwrap();
49             }
50         }
51
52         // Ritorno l'ordine di arrivo calcolato all'ingresso
53         rank
54     }
55 }

```